

AI534 — Written Homework Assignment 1 (30 pts + 6 bonus pts)

This written assignment covers the contents of linear regression and logistic regression. The key concepts covered here include:

- Maximum likelihood estimation (MLE)
- Gradient descent learning
- Decision theory for probabilistic classifiers
- Maximum A Posteriori (MAP) parameter estimation
- Perceptron

1. MLE for uniform distribution. [3pt]

Given a set of IID observed samples $x_1, \dots, x_n \sim \text{uniform}(0, \theta)$, we wish to estimate the parameter θ .

- (a) (1 pt) Write down the likelihood function of θ .

The probability density function (PDF) of a uniform distribution $U(0, \theta)$ is:

$$f(x|\theta) = \begin{cases} \frac{1}{\theta} & 0 \leq x \leq \theta, \\ 0 & \text{otherwise.} \end{cases}$$

Suppose we have n independent and identically distributed (IID) samples, $x_1, \dots, x_n$. The likelihood function $L(\theta)$ represents how likely the data is for a given parameter θ . To calculate it, we multiply the probability densities for each individual sample.

$$L(\theta) = \prod_{i=1}^n f(x_i|\theta).$$

Since each x_i follows the uniform distribution $U(0, \theta)$, the likelihood function becomes:

$$L(\theta) = \prod_{i=1}^n \frac{1}{\theta} = \frac{1}{\theta^n}, \quad \text{for } \max(x_i) \leq \theta.$$

If θ is less than any of the samples, the likelihood is 0. Therefore the likelihood function is:

$$L(\theta) = \frac{1}{\theta^n} \quad \text{if } \theta \geq \max(x_1, \dots, x_n), \quad L(\theta) = 0 \text{ otherwise.}$$

- (b) (2 pts) Derive the maximum likelihood estimation for θ , which is the value for *theta* that maximizes the function of part (a). (Hint: The likelihood function is a monotonic function. So the maximizing solution is at the extreme— there is no need to take derivative for this case.)

To find the maximum likelihood estimate for θ , we need to maximize the likelihood function $L(\theta)$. Since the likelihood function $L(\theta) = \frac{1}{\theta^n}$ is a decreasing (monotonic) function of θ , the maximum likelihood occurs when θ is as small as possible, subject to the constraint that $\theta \geq \max(x_1, \dots, x_n)$. So the maximum likelihood estimate (MLE) of θ is:

$$\hat{\theta} = \max(x_1, \dots, x_n)$$

This value for θ maximizes the likelihood function so the MLE for θ is the largest observed value in the sample.

2. **Weighted linear regression.** [10pt] In class when discussing linear regression, we assume that the Gaussian noise is iid (identically independently distributed). In practice, we may have some extra information regarding the fidelity of each data point. For example, we may know that some examples have higher noise variance than others. To model this, we can model the noise variable $\epsilon_1, \epsilon_2, \dots, \epsilon_n$ as distinct Gaussian's, i.e., $\epsilon_i \sim N(0, \sigma_i^2)$ with known variance σ_i^2 . How will this influence our linear regression model? Let's work it out.

- (a) (3pts) Write down the log likelihood function of $\mathbf{w}$ under this new modeling assumption.

Given that the likelihood function for the weighted linear regression model is:

$$L(\mathbf{w}) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(y_i - \mathbf{x}_i^T \mathbf{w})^2}{2\sigma_i^2}\right)$$

*so the **log likelihood function** can be written by taking the logarithm of the likelihood function $L(\mathbf{w})$:*

$$\log L(\mathbf{w}) = \log \left(\prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(y_i - \mathbf{x}_i^T \mathbf{w})^2}{2\sigma_i^2}\right) \right)$$

Since the log of a product is the sum of the logs we will get -

$$\log L(\mathbf{w}) = \sum_{i=1}^n \log \left(\frac{1}{\sqrt{2\pi\sigma_i^2}} \right) + \sum_{i=1}^n \log \left(\exp\left(-\frac{(y_i - \mathbf{x}_i^T \mathbf{w})^2}{2\sigma_i^2}\right) \right)$$

Simplifying this further

$$\begin{aligned} \log L(\mathbf{w}) &= -\sum_{i=1}^n \log \left(\sqrt{2\pi\sigma_i^2} \right) - \sum_{i=1}^n \frac{(y_i - \mathbf{x}_i^T \mathbf{w})^2}{2\sigma_i^2} \\ \log L(\mathbf{w}) &= -\sum_{i=1}^n \left(\frac{1}{2} \log(2\pi\sigma_i^2) \right) - \sum_{i=1}^n \frac{(y_i - \mathbf{x}_i^T \mathbf{w})^2}{2\sigma_i^2} \\ \log L(\mathbf{w}) &= -\frac{1}{2} \sum_{i=1}^n \log(2\pi\sigma_i^2) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mathbf{x}_i^T \mathbf{w})^2}{\sigma_i^2} \end{aligned}$$

This is the log likelihood function of $\mathbf{w}$ under the new modeling assumption with heteroscedastic noise.

- (b) (1pts) Show that maximizing the log likelihood is equivalent to minimizing a **weighted square loss function** $J(\mathbf{W}) = \sum_{i=1}^n a_i (\mathbf{w}^T \mathbf{x}_i - y_i)^2$, and express each a_i in terms of σ_i .

To estimate the parameters w , we look for the values that make the data fit best by maximizing the log of the likelihood function.

$$\log L(w) = -\frac{1}{2} \sum_{i=1}^n \log(2\pi\sigma_i^2) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mathbf{x}_i^T w)^2}{\sigma_i^2}$$

Since the part of the equation, $-\frac{1}{2} \sum_{i=1}^n \log(2\pi\sigma_i^2)$, is independent of w , we can ignore it when optimizing with respect to w so we just have the second term to minimize:

$$\min_w \sum_{i=1}^n \frac{(y_i - \mathbf{x}_i^T w)^2}{\sigma_i^2}$$

Comparing with a weighted square loss function The previous expression can be compared to a weighted square loss function of the form:

$$J(w) = \sum_{i=1}^n a_i (x_i^T w - y_i)^2$$

To match the form, we can see that the weights a_i must be:

$$a_i = \frac{1}{\sigma_i^2}$$

Therefore minimizing the log likelihood is equivalent to minimizing the weighted square loss function, where each data point is weighted by the inverse of its noise variance.

Maximizing the log likelihood for this weighted linear regression model is the same as minimizing a weighted sum of squared errors, with the weights given by

$$a_i = \frac{1}{\sigma_i^2}$$

This result shows that data points with higher noise variance (meaning less reliable data) have less impact on the overall objective, while data points with lower noise variance (more reliable data) carry more weight in influencing the parameter estimates.

- (c) (3 pts) Take the gradient of the loss function $J(\mathbf{w})$ and provide the batch gradient descent update rule for optimizing $\mathbf{w}$.

The weighted loss function is

$$J(\mathbf{w}) = \sum_{i=1}^n a_i (\mathbf{x}_i^T \mathbf{w} - y_i)^2$$

Here, $a_i = \frac{1}{\sigma_i^2}$ represents the weights assigned to each data point, where σ_i^2 is the noise variance for that data point.

The loss for each data point i is:

$$a_i (\mathbf{x}_i^T \mathbf{w} - y_i)^2$$

Summing over all data points, the total loss function becomes:

$$J(\mathbf{w}) = \sum_{i=1}^n a_i (\mathbf{x}_i^T \mathbf{w} - y_i)^2$$

Computing the gradient of the loss function - Considering the individual term for a single data point i :

$$a_i (\mathbf{x}_i^T \mathbf{w} - y_i)^2$$

The gradient with respect to $\mathbf{w}$ is:

$$\nabla_{\mathbf{w}} [a_i (\mathbf{x}_i^T \mathbf{w} - y_i)^2] = a_i \cdot 2(\mathbf{x}_i^T \mathbf{w} - y_i) \cdot \mathbf{x}_i$$

because the derivative of $(\mathbf{x}_i^T \mathbf{w} - y_i)^2$ with respect to $\mathbf{w}$ is $2(\mathbf{x}_i^T \mathbf{w} - y_i)\mathbf{x}_i$.

Now, when we sum over all data points, the gradient of the total loss function is:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \sum_{i=1}^n a_i \cdot 2(\mathbf{x}_i^T \mathbf{w} - y_i) \cdot \mathbf{x}_i$$

Simplifying:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 2 \sum_{i=1}^n a_i (\mathbf{x}_i^T \mathbf{w} - y_i) \mathbf{x}_i$$

The batch gradient descent algorithm updates the weight vector $\mathbf{w}$ iteratively by moving in the direction of the negative gradient. The update rule for $\mathbf{w}$ is:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta \nabla_{\mathbf{w}} J(\mathbf{w})$$

where η is the learning rate, and $\nabla_{\mathbf{w}} J(\mathbf{w})$ is the gradient of the loss function with respect to $\mathbf{w}$. Substituting the expression for the gradient:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta \cdot 2 \sum_{i=1}^n a_i (\mathbf{x}_i^T \mathbf{w} - y_i) \mathbf{x}_i$$

Simplifying it further:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - 2\eta \sum_{i=1}^n a_i (\mathbf{x}_i^T \mathbf{w} - y_i) \mathbf{x}_i$$

Final Batch Gradient Descent Update Rule

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - 2\eta \sum_{i=1}^n a_i (\mathbf{x}_i^T \mathbf{w} - y_i) \mathbf{x}_i$$

Here, η is the learning rate, and $a_i = \frac{1}{\sigma_i^2}$ are the weights that adjust for the noise variance of each data point. This rule will iteratively update the weight vector $\mathbf{w}$ to minimize the weighted square loss function.

- (d) (3 pts) Derive a closed form solution to this optimization problem. Hint: begin by rewrite the objective into matrix form using a diagonal matrix $\mathbf{A}$ with $A(i, i) = a_i$.

*Deriving a closed-form solution for the weighted linear regression problem -
The loss function is:*

$$J(\mathbf{w}) = \sum_{i=1}^n a_i (\mathbf{x}_i^T \mathbf{w} - y_i)^2$$

where $a_i = \frac{1}{\sigma_i^2}$. To find a closed-form solution, we need to express this loss function in matrix form and minimize it.

Expressing the loss function in matrix form

We can represent the loss function in matrix form using the following definitions:

- $\mathbf{X} \in \mathbb{R}^{n \times d}$ is the matrix of features, where each row is a feature vector $\mathbf{x}_i$.
- $\mathbf{w} \in \mathbb{R}^d$ is the weight vector we are solving for.
- $\mathbf{y} \in \mathbb{R}^n$ is the vector of outputs, where each element is y_i .
- $\mathbf{A} \in \mathbb{R}^{n \times n}$ is a diagonal matrix, where $\mathbf{A}(i, i) = a_i = \frac{1}{\sigma_i^2}$.

Using these definitions, the weighted sum of squared residuals can be written in matrix form as:

$$J(\mathbf{w}) = (\mathbf{X}\mathbf{w} - \mathbf{y})^T \mathbf{A} (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Next we can expand it in the quadratic form

$$J(\mathbf{w}) = (\mathbf{X}\mathbf{w} - \mathbf{y})^T \mathbf{A} (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Expanding this:

$$J(\mathbf{w}) = (\mathbf{w}^T \mathbf{X}^T - \mathbf{y}^T) \mathbf{A} (\mathbf{X} \mathbf{w} - \mathbf{y})$$

$$J(\mathbf{w}) = \mathbf{w}^T \mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w} - 2 \mathbf{y}^T \mathbf{A} \mathbf{X} \mathbf{w} + \mathbf{y}^T \mathbf{A} \mathbf{y}$$

Next step is to minimize the objective To find the value of $\mathbf{w}$ that minimizes the objective, we take the gradient of $J(\mathbf{w})$ with respect to $\mathbf{w}$ and set it equal to zero. The gradient of $J(\mathbf{w})$ is:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 2 \mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w} - 2 \mathbf{X}^T \mathbf{A} \mathbf{y}$$

Setting the gradient equal to zero to minimize $J(\mathbf{w})$:

$$2 \mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w} = 2 \mathbf{X}^T \mathbf{A} \mathbf{y}$$

Simplifying:

$$\mathbf{X}^T \mathbf{A} \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{A} \mathbf{y}$$

Next we solve for $\mathbf{w}$ To solve for $\mathbf{w}$ we need to multiply both sides by the inverse of $\mathbf{X}^T \mathbf{A} \mathbf{X}$ (assuming it is invertible):

$$\mathbf{w} = (\mathbf{X}^T \mathbf{A} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{A} \mathbf{y}$$

This is the closed-form solution for the weighted linear regression problem.

The closed-form solution for $\mathbf{w}$ is:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{A} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{A} \mathbf{y}$$

where:

- $\mathbf{X}$ is the feature matrix,
- $\mathbf{A}$ is the diagonal matrix with $a_i = \frac{1}{\sigma_i^2}$,
- $\mathbf{y}$ is the output vector.

3. Decision theory: working with expectations. [6pt]

In this problem, you will analyze a scenario where the Maximum A-Posteriori (MAP) decision rule, which you learned in class, is not appropriate. Instead, you'll explore how to make decisions based on minimizing expected costs.

Consider a spam filter that predicts whether an email is spam, using probabilistic predictions. For this filter, there are costs associated with making errors (misclassifying emails), but these costs are not symmetric. Misclassifying a non-spam email as spam (i.e., filtering out an important email) is more costly than misclassifying a spam email as non-spam.

The following table shows the cost of each possible outcome:

predicted label $\hat{y}$	true label y	
	non-spam	spam
non-spam	0	1
spam	10	0

Table 1: A mis-classification cost matrix for the spam filter problem.

- If the filter's prediction is correct, there is no cost.
- If a non-spam email is classified as spam, there is a cost of 10.
- If a spam email is classified as non-spam, there is a cost of 1.

Here we will go through some questions to help you figure out how to use the probability and misclassification costs to make predictions.

- (a) (2 pt) You received an email for which the spam filter predicts that it is a spam with $p = 0.8$. We want to make the decision that minimizes *the expected cost*.

Question: Should you classify this particular email as spam or non-spam? [Hint: Compare the expected cost of classifying the email as spam versus non-spam. Choose the classification that results in the lower expected cost.]

$$\text{Expected cost of predicting non-spam} = P(\text{spam}) \cdot 1 + P(\text{non-spam}) \cdot 0$$

$$\text{Expected cost of predicting non-spam} = 0.8 \cdot 1 + 0.2 \cdot 0 = 0.8$$

$$\text{Expected cost of predicting spam} = P(\text{non-spam}) \cdot 10 + P(\text{spam}) \cdot 0$$

$$P(\text{non-spam}) = 0.2$$

$$P(\text{spam}) = 0.8$$

$$\text{Expected cost of predicting spam} = 0.2 \cdot 10 + 0.8 \cdot 0 = 2$$

Comparing the expected costs of two decisions

$$\text{Expected cost of predicting non-spam} = 0.8$$

$$\text{Expected cost of predicting spam} = 2$$

Since it's cheaper to predict an email as non-spam compared to spam, it makes sense to classify the email as non-spam. The expected cost of marking it as non-spam is 0.8, which is much lower than the cost of labelling it as spam, which is 2.

- (b) (2 pts) The MAP decision rule would classify an email as spam if $p > 0.5$, but this rule does not minimize expected cost in this case. We need a new rule that compares p to a different threshold θ . The value of θ should be chosen to minimize the expected cost based on the costs in the table.

Question: What is the value of θ that works for the costs specified in Table 1? [Hint: To find the threshold θ , set up the decision rule by comparing the expected cost of each decision, as you did in (a), then Solve for p in terms of the costs.]

To find the threshold θ for the spam filter, such that the decision rule minimizes the expected cost based on the misclassification costs provided in Table 1.

In part (a), we compared the expected costs of classifying an email as either non-spam or spam. Now, we will derive the threshold θ by comparing the expected costs and solving for the point where they are equal.

Expected Cost of Predicting Non-Spam

As calculated in part (a), the expected cost of predicting non-spam is:

$$\text{Expected cost of predicting non-spam} = P(\text{spam}) \cdot 1 + P(\text{non-spam}) \cdot 0$$

By simplifying it further we get:

$$\text{Expected cost of predicting non-spam} = P(\text{spam})$$

Expected Cost of Predicting Spam

$$\text{Expected cost of predicting spam} = P(\text{non-spam}) \cdot 10 + P(\text{spam}) \cdot 0$$

By simplifying it further we get:

$$\text{Expected cost of predicting spam} = 10 \cdot P(\text{non-spam}) = 10 \cdot (1 - P(\text{spam}))$$

Setting up the Decision Rule

To minimize the expected cost, we should choose the classification with the lower expected cost so the decision rule becomes:

$$\text{Predict non-spam if: } P(\text{spam}) < 10 \cdot (1 - P(\text{spam}))$$

Simplifying the inequality:

$$P(\text{spam}) < 10 - 10 \cdot P(\text{spam})$$

$$11 \cdot P(\text{spam}) < 10$$

$$P(\text{spam}) < \frac{10}{11}$$

So the threshold θ is $\frac{10}{11}$.

Final Decision Rule

Under the typical MAP rule, we would classify an email as spam if $P(\text{spam}) > 0.5$ but to minimize the expected cost given the cost matrix, we should classify the email as spam only if $P(\text{spam}) \geq \frac{10}{11}$. The threshold θ that minimizes the expected cost based on the specified cost matrix is:

$$\theta = \frac{10}{11} \approx 0.909$$

- (c) (2pts) Now, imagine that the optimal decision rule would use $\theta = 1/5$ as the threshold for classifying an email as spam. **Question:** Can you provide a new cost table where this would be the case? [Hint: Use the relationship between the costs and θ that you derived in part (b). Based on this relationship, adjust the misclassification costs in the table to achieve $\theta = 1/5$.]

Deriving a New Cost Table for $\theta = \frac{1}{5}$

The threshold was determined in the previous part by the following relationship between the costs:

$$\theta = \frac{C(\text{non-spam as spam})}{C(\text{spam as non-spam}) + C(\text{non-spam as spam})}$$

Where:

- $C(\text{non-spam as spam})$ is the cost of classifying a non-spam email as spam.
- $C(\text{spam as non-spam})$ is the cost of classifying a spam email as non-spam.

For $\theta = \frac{1}{5}$ using the same relationship as derived in part (b):

$$\theta = \frac{C(\text{non-spam as spam})}{C(\text{spam as non-spam}) + C(\text{non-spam as spam})}$$

Substituting $\theta = \frac{1}{5}$ into the equation:

$$\frac{1}{5} = \frac{C(\text{non-spam as spam})}{C(\text{spam as non-spam}) + C(\text{non-spam as spam})}$$

Multiply both sides by $C(\text{spam as non-spam}) + C(\text{non-spam as spam})$:

$$C(\text{spam as non-spam}) + C(\text{non-spam as spam}) = 5 \cdot C(\text{non-spam as spam})$$

By rearranging we get:

$$C(\text{spam as non-spam}) = 4 \cdot C(\text{non-spam as spam})$$

To create the New Cost Table

This relationship tells us that the cost of misclassifying spam as non-spam should be 4 times the cost of misclassifying non-spam as spam.

Assigning a new cost to $C(\text{non-spam as spam})$. Assuming we choose $C(\text{non-spam as spam}) = 1$. Then we get:

$$C(\text{spam as non-spam}) = 4 \cdot 1 = 4$$

This gives us the following cost table:

Predicted Label $\hat{y}$	True Label y	Non-Spam	Spam
Non-Spam	Non-Spam	0	4
Spam	Spam	1	0

Verifying the New Threshold $\theta = \frac{1}{5}$

To verify that this new cost table corresponds to $\theta = \frac{1}{5}$, we can use the formula again:

$$\theta = \frac{C(\text{non-spam as spam})}{C(\text{spam as non-spam}) + C(\text{non-spam as spam})} = \frac{1}{4 + 1} = \frac{1}{5}$$

this cost table satisfies $\theta = \frac{1}{5}$.

The new cost table that results in $\theta = \frac{1}{5}$ is:

Predicted Label $\hat{y}$	True Label y	Non-Spam	Spam
Non-Spam	Non-Spam	0	4
Spam	Spam	1	0

4. **Maximum A-Posteriori Estimation.** [8pt] Suppose we observe the values of n IID random variables $X_1, \dots, X_n$ drawn from a single Bernoulli distribution with parameter θ . In other words, for each X_i , we know that $P(X_i = 1) = \theta$ and $P(X_i = 0) = 1 - \theta$. In the Bayesian framework, we treat θ as a random variable, and use a prior probability distribution over θ to express our prior knowledge/preference about θ . In this framework, $X_1, \dots, X_n$ can be viewed as generated by:

- First, the value of θ is drawn from a given prior probability distribution
- Second, $X_1, \dots, X_n$ are drawn independently from a Bernoulli distribution with this θ value.

In this setting, Maximum A-Posteriori (MAP) estimation is a way to estimate θ by finding the value that maximizes the posterior probability, given both its prior distribution and the observed data. The MAP estimation of θ is given by:

$$\hat{\theta}_{MAP} = \underset{\hat{\theta}}{\operatorname{argmax}} P(\theta = \hat{\theta} | X_1, \dots, X_n)$$

By applying Bayes' theorem, this becomes:

$$\hat{\theta}_{MAP} = \underset{\hat{\theta}}{\operatorname{argmax}} P(X_1, \dots, X_n | \theta = \hat{\theta}) P(\theta = \hat{\theta}) = \underset{\hat{\theta}}{\operatorname{argmax}} L(\hat{\theta}) p(\hat{\theta})$$

where $L(\hat{\theta})$ is the likelihood function of the data given θ , and $p(\hat{\theta})$ is the prior distribution over θ .

Now consider using a beta distribution as the prior: $\theta \sim \text{Beta}(\alpha, \beta)$, whose PDF function is

$$p(\hat{\theta}) = \frac{\hat{\theta}^{(\alpha-1)}(1-\hat{\theta})^{(\beta-1)}}{B(\alpha, \beta)}$$

where $B(\alpha, \beta)$ is a normalizing constant.

- (a) (3 pts) Derive the posterior distribution $p(\hat{\theta} | X_1, \dots, X_n, \alpha, \beta)$. Compare the form of the posterior distribution with that of the beta distribution, you will see the posterior is also a beta distribution. What the updated α and β parameters for the posterior?

Deriving the posterior distribution for θ , given the observed data $X_1, X_2, \dots, X_n$ and the prior distribution $\theta \sim \text{Beta}(\alpha, \beta)$.

*In Bayesian statistics, the posterior distribution is the probability distribution of a parameter (in this case, θ) after taking into account both the prior distribution and the likelihood of the observed data. This is done using **Bayes' theorem**:*

$$p(\theta | X_1, \dots, X_n) = \frac{P(X_1, \dots, X_n | \theta) P(\theta)}{P(X_1, \dots, X_n)}$$

Where:

- $P(X_1, \dots, X_n | \theta)$ is the likelihood of the data given θ ,
- $P(\theta)$ is the prior distribution of θ ,
- $P(X_1, \dots, X_n)$ is the marginal likelihood (or evidence), which normalizes the posterior distribution.

Likelihood Function Since $X_1, \dots, X_n$ are IID random variables drawn from a Bernoulli distribution with parameter θ , the likelihood function is:

$$L(\theta) = P(X_1, \dots, X_n | \theta) = \prod_{i=1}^n \theta^{X_i} (1-\theta)^{1-X_i}$$

Let $S = \sum_{i=1}^n X_i$, the number of successes in the observed data. The likelihood can then be written as:

$$L(\theta) = \theta^S (1-\theta)^{n-S}$$

This is the likelihood of the data given θ .

Prior Distribution We are given that the prior distribution for θ is a Beta distribution with parameters α and β :

$$p(\theta) = \frac{\theta^{\alpha-1} (1-\theta)^{\beta-1}}{B(\alpha, \beta)}$$

where $B(\alpha, \beta)$ is the Beta function, which serves as a normalizing constant.

Applying Bayes' Theorem to derive the posterior distribution:

$$p(\theta | X_1, \dots, X_n) = \frac{P(X_1, \dots, X_n | \theta) P(\theta)}{P(X_1, \dots, X_n)}$$

Substituting the likelihood function and the earlier distribution:

$$p(\theta|X_1, \dots, X_n) \propto \theta^S (1 - \theta)^{n-S} \cdot \theta^{\alpha-1} (1 - \theta)^{\beta-1}$$

Simplifying it further we get:

$$p(\theta|X_1, \dots, X_n) \propto \theta^{S+\alpha-1} (1 - \theta)^{n-S+\beta-1}$$

This is the unnormalized posterior distribution. The normalizing constant, which make sures that the total probability integrates to 1, will be the Beta function with updated parameters.

Posterior Distribution as a Beta Distribution

The expression for the posterior distribution:

$$p(\theta|X_1, \dots, X_n) \propto \theta^{S+\alpha-1} (1 - \theta)^{n-S+\beta-1}$$

is exactly the form of a Beta distribution with parameters α' and β' , where:

$$\alpha' = S + \alpha$$

$$\beta' = n - S + \beta$$

Thus, the posterior distribution $p(\theta|X_1, \dots, X_n)$ is a Beta distribution with updated parameters:

$$\theta|X_1, \dots, X_n \sim \text{Beta}(S + \alpha, n - S + \beta)$$

The updated parameters α' and β' for the posterior distribution are:

- $\alpha' = S + \alpha$, where $S = \sum_{i=1}^n X_i$ is the number of successes (i.e., the number of times $X_i = 1$),
- $\beta' = n - S + \beta$, where $n - S$ is the number of failures (i.e., the number of times $X_i = 0$).

Comparison with the Prior

The prior distribution for θ was $\text{Beta}(\alpha, \beta)$. After observing the data $X_1, \dots, X_n$, the posterior distribution remains in the Beta family, but the parameters are updated to $\alpha' = S + \alpha$ and $\beta' = n - S + \beta$.

This shows that the Beta distribution is a **conjugate prior** for the Bernoulli distribution, meaning that the posterior distribution has the same form as the prior distribution, with updated parameters.

The posterior distribution $p(\theta|X_1, \dots, X_n)$ is a Beta distribution:

$$\theta|X_1, \dots, X_n \sim \text{Beta}(S + \alpha, n - S + \beta)$$

Where:

- $\alpha' = S + \alpha$ is the updated shape parameter for successes,
- $\beta' = n - S + \beta$ is the updated shape parameter for failures.

The posterior distribution remains a Beta distribution, but with parameters that are adjusted based on the observed data.

- (b) (2 pts) Suppose we use $\text{Beta}(2, 2)$ as the prior, what Beta distribution do we get for the posterior after we observe 5 coin tosses and 2 of them are head? What is the posterior distribution of θ after we observe 50 coin tosses and 20 of them are head? (you don't need to write out the distributions, simply provide the α and β distribution would suffice.

The prior distribution for θ is given as $\theta \sim \text{Beta}(2, 2)$. This means that:

- $\alpha = 2$

- $\beta = 2$

To compute the posterior distribution, we use the formula for the updated parameters of the Beta distribution based on the number of successes S and failures $n - S$ observed in the data.

The general update rule for the posterior parameters is:

$$\alpha' = S + \alpha$$

$$\beta' = n - S + \beta$$

Where:

- S is the number of successes (heads in the coin tosses),
- n is the total number of coin tosses (trials),
- $n - S$ is the number of failures (tails in the coin tosses).

Case 1: 5 Coin Tosses, 2 Heads

Here, we have:

- $n = 5$ (total coin tosses),
- $S = 2$ (number of heads, or successes),
- $n - S = 3$ (number of tails, or failures).

Using the update rules for the Beta distribution:

$$\alpha' = S + \alpha = 2 + 2 = 4$$

$$\beta' = n - S + \beta = 3 + 2 = 5$$

Thus, after observing 5 coin tosses with 2 heads, the posterior distribution is $\text{Beta}(4, 5)$.

Case 2: 50 Coin Tosses, 20 Heads

Here, we have:

- $n = 50$ (total coin tosses),
- $S = 20$ (number of heads, or successes),
- $n - S = 30$ (number of tails, or failures).

Using the update rules for the Beta distribution:

$$\alpha' = S + \alpha = 20 + 2 = 22$$

$$\beta' = n - S + \beta = 30 + 2 = 32$$

After observing 50 coin tosses with 20 heads, the posterior distribution is $\text{Beta}(22, 32)$.

- After 5 coin tosses with 2 heads, the posterior distribution is $\text{Beta}(4, 5)$.
- After 50 coin tosses with 20 heads, the posterior distribution is $\text{Beta}(22, 32)$.

- (c) (1pt) Plot the pdf function of the prior $\text{Beta}(2, 2)$ and the two posterior distributions. You can use any software (e.g., R, Python, Matlab) for this plot.

Posterior Distributions for Beta(2, 2) Prior Below is the plot of the prior $\text{Beta}(2, 2)$ distribution and the two posterior distributions after observing 5 coin tosses (with 2 heads) and 50 coin tosses (with 20 heads).

As more data is observed, the posterior distribution becomes more concentrated around the estimated value of θ , reflecting increased certainty about the parameter.

image.png

- (d) (2pt) Assume that $\theta = 0.4$ is the true probability, as we observe more and more coin tosses from this coin, how would the shape of the posterior change as more data is observed? Will the MAP estimate converge toward the true value?

Concentration Around the True Value: As more coin tosses are observed, the posterior distribution becomes narrower and more concentrated around $\theta = 0.4$. Initially, with limited data, the posterior is uncertain and spread out, but as observations increase, the effect of the prior diminishes, and the posterior is dominated by the data.

MAP Estimate Converging to the True Value: The MAP estimate, which is the mode of the posterior distribution, will converge to $\theta = 0.4$ as more data is observed. This happens because the posterior becomes increasingly centered on $\theta = 0.4$ and reflects the true value more accurately.

- With more data, the posterior distribution becomes more concentrated around $\theta = 0.4$.
- The MAP estimate will converge to $\theta = 0.4$, becoming more accurate as observations increase.

This is a key feature of Bayesian inference: with enough data, the MAP estimate converges to the true parameter value.

5. **Perceptron.** [3pt] Assume a data set consists only of a single data point $\{(x, +1)\}$. How many times would the Perceptron algorithm mis-classify this point $\mathbf{x}$ before convergence? What if the initial weight vector $\mathbf{w}_0$ was initialized randomly and not as the all-zero vector? Derive the number of times as a function of $\mathbf{w}_0$ and $\mathbf{x}$.

- (a) (1 pts) Case 1: $\mathbf{w}_0 = 0$.

The Perceptron updates its weight vector as follows:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + y \cdot \mathbf{x}$$

Initially, with $\mathbf{w}_0 = 0$, the prediction is:

$$\hat{y} = \text{sign}(\mathbf{w}_0^T \mathbf{x}) = 0$$

Since the true label is $+1$, this is a misclassification, and the algorithm updates:

$$\mathbf{w}_1 = \mathbf{x}$$

On the next iteration, the prediction is:

$$\hat{y} = \text{sign}(\mathbf{w}_1^T \mathbf{x}) = 1$$

The algorithm converges after one misclassification.

The Perceptron misclassifies once when $\mathbf{w}_0 = 0$.

(b) (2 pts) Case 2: $\mathbf{w}_0 \neq 0$:

The Perceptron predicts based on:

$$\hat{y} = \text{sign}(\mathbf{w}_0^T \mathbf{x})$$

Two possibilities:

- $\mathbf{w}_0^T \mathbf{x} > 0$: The prediction is correct, no misclassification occurs.
- $\mathbf{w}_0^T \mathbf{x} \leq 0$: The prediction is incorrect, and the weight is updated:

$$\mathbf{w}_1 = \mathbf{w}_0 + \mathbf{x}$$

On the next iteration:

$$\hat{y} = \text{sign}((\mathbf{w}_0 + \mathbf{x})^T \mathbf{x}) = 1$$

The algorithm converges after one misclassification.

If $\mathbf{w}_0^T \mathbf{x} > 0$, no misclassification occurs. If $\mathbf{w}_0^T \mathbf{x} \leq 0$, the Perceptron misclassifies once before convergence.

6. **Bonus: MLE for multi-class logistic regression.** [6 pts] Consider the maximum likelihood estimation problem for multi-class logistic regression using the soft-max function defined below:

$$p(y = k | \mathbf{x}) = \frac{\exp(\mathbf{w}_k^T \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x})}$$

We can write out the likelihood function as:

$$L(\mathbf{w}) = \prod_{i=1}^N \prod_{k=1}^K p(y = k | \mathbf{x}_i)^{y_{ik}}$$

where y_{ik} is an indicator variable taking value 1 if $y_i = k$.

(a) (1 pts) Provide the log-likelihood function.

Log-likelihood function for the multi-class logistic regression problem:

$$\log L(\mathbf{w}) = \sum_{i=1}^N \sum_{k=1}^K y_{ik} \left(\mathbf{w}_k^T \mathbf{x}_i - \log \left(\sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x}_i) \right) \right)$$

Where:

- N is the number of data points,
- K is the number of classes,
- $\mathbf{w}_k$ is the weight vector for class k ,
- y_{ik} is an indicator variable that is 1 if the true label of $\mathbf{x}_i$ is class k , and 0 otherwise,
- $\mathbf{x}_i$ is the feature vector for the i -th data point.

- (b) (5 pts) Derive the gradient of the log-likelihood function w.r.t the weight vector $\mathbf{w}_c$ of class c . [Hint: the solution to this problem is provided in the Logistic regression lecture slide. You just need to fill in the missing derivation. Note that for any example $\mathbf{x}_i$, the denominator in the softmax function $\sum_j \exp(\mathbf{w}_j^T \mathbf{x}_i)$ is the same for all k — denoting it as z_i makes it simpler to work through the derivation, but be sure to remember that z_i is a function of all $\mathbf{w}_k$'s.]

Computing the gradient of the log-likelihood function with respect to the weight vector $\mathbf{w}_c$ for class c . The log-likelihood function for multi-class logistic regression is given by:

$$\log L(\mathbf{w}) = \sum_{i=1}^N \sum_{k=1}^K y_{ik} \left(\mathbf{w}_k^T \mathbf{x}_i - \log \left(\sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x}_i) \right) \right)$$

Where:

- N is the number of data points,
- K is the number of classes,
- $\mathbf{x}_i \in \mathbb{R}^d$ is the feature vector for the i -th example,
- $\mathbf{w}_k \in \mathbb{R}^d$ is the weight vector for class k ,
- y_{ik} is an indicator variable that is 1 if the true label of $\mathbf{x}_i$ is class k , and 0 otherwise.

shorthand notation for the denominator in the softmax function:

$$z_i = \sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x}_i)$$

So the probability of class k for the i -th example will become:

$$p(y = k | \mathbf{x}_i) = \frac{\exp(\mathbf{w}_k^T \mathbf{x}_i)}{z_i}$$

Log-Likelihood for Class c

Taking the derivative of the log-likelihood function with respect to $\mathbf{w}_c$, the weight vector for class c . For simplicity, let's isolate the terms involving $\mathbf{w}_c$:

$$\log L(\mathbf{w}) = \sum_{i=1}^N [y_{ic} \mathbf{w}_c^T \mathbf{x}_i - \log(z_i)]$$

Derivative of the Log-Likelihood

Now, we differentiate the log-likelihood with respect to $\mathbf{w}_c$.

1. Derivative of the first term $y_{ic} \mathbf{w}_c^T \mathbf{x}_i$:

$$\frac{\partial}{\partial \mathbf{w}_c} (y_{ic} \mathbf{w}_c^T \mathbf{x}_i) = y_{ic} \mathbf{x}_i$$

Derivative of the second term $\log z_i$:

Since $z_i = \sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x}_i)$, and only $\mathbf{w}_c$ is affected in this derivative, we use the chain rule. The derivative of $\log z_i$ is:

$$\frac{\partial}{\partial \mathbf{w}_c} \log z_i = \frac{1}{z_i} \cdot \frac{\partial z_i}{\partial \mathbf{w}_c}$$

Now, the derivative of z_i with respect to $\mathbf{w}_c$ is:

$$\frac{\partial z_i}{\partial \mathbf{w}_c} = \frac{\partial}{\partial \mathbf{w}_c} \sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x}_i) = \exp(\mathbf{w}_c^T \mathbf{x}_i) \mathbf{x}_i$$

So the derivative of $\log z_i$ with respect to $\mathbf{w}_c$ is:

$$\frac{\partial}{\partial \mathbf{w}_c} \log z_i = \frac{\exp(\mathbf{w}_c^T \mathbf{x}_i)}{z_i} \mathbf{x}_i = p(y = c | \mathbf{x}_i) \mathbf{x}_i$$

Gradient of the Log-Likelihood

Putting it all together, the gradient of the log-likelihood with respect to $\mathbf{w}_c$ is:

$$\frac{\partial \log L(\mathbf{w})}{\partial \mathbf{w}_c} = \sum_{i=1}^N (y_{ic} \mathbf{x}_i - p(y = c | \mathbf{x}_i) \mathbf{x}_i)$$

Where:

- y_{ic} is 1 if the true label of $\mathbf{x}_i$ is class c , and 0 otherwise,
- $p(y = c | \mathbf{x}_i)$ is the predicted probability that the label is c for the feature vector $\mathbf{x}_i$.

The gradient of the log-likelihood with respect to the weight vector $\mathbf{w}_c$ is:

$$\frac{\partial \log L(\mathbf{w})}{\partial \mathbf{w}_c} = \sum_{i=1}^N \mathbf{x}_i (y_{ic} - p(y = c | \mathbf{x}_i))$$

This gradient is used to update the weight vector for class c during optimization (e.g., using gradient ascent or descent).